

RÉPUBLIQUE TUNISIENNE
UNIVERSITÉ PRIVÉE DE TUNISIE



Rapport de Stage d'Été

Développement d'une plateforme intelligente de détection de fuites d'eau basée sur l'IoT et l'intelligence artificielle

Application Droppy

Water Leak Detection & Anomaly Analysis Suite

Entreprise d'accueil : SFM Connect – Groupe SFM, Tunis

Élaboré par : Aydi ALA

Encadrant professionnel : Mr. Fadi BOURICHA

Année universitaire : 2025 / 2026

Remerciements

Je tiens à exprimer ma profonde gratitude à l'équipe de **SFM Connect**, branche du Groupe SFM, pour m'avoir accueilli et accompagné tout au long de ce stage.

Cette expérience m'a permis de mettre en pratique mes connaissances et de développer de nouvelles compétences dans les domaines du développement logiciel, de l'IoT et de l'intelligence artificielle.

Je remercie tout particulièrement **Mr. Fadi Bouricha**, mon encadrant professionnel, pour son accompagnement, ses conseils, sa disponibilité et la confiance accordée tout au long de la réalisation de ce projet.

Mes remerciements s'adressent également à l'ensemble des membres de l'équipe pour leur disponibilité, leurs conseils techniques et les échanges qui ont contribué à enrichir mon expérience professionnelle.

Enfin, j'adresse mes sincères remerciements à mes enseignants ainsi qu'à ma famille et à mes proches pour leur soutien et leurs encouragements durant cette période.

Résumé

Ce rapport présente la conception et la réalisation de **Droppy**, une suite logicielle multi-services destinée à la détection automatique de fuites d'eau et d'anomalies dans les réseaux de distribution.

Développée au sein de SFM Connect, la solution combine des données issues de capteurs IoT, des règles métier hydrauliques telles que le *Minimum Night Flow* et la détection d'incohérences entre l'état des vannes et le débit, ainsi que des méthodes d'apprentissage automatique, notamment **Isolation Forest** et un modèle de prévision de consommation.

L'architecture repose sur trois composants principaux : un microservice dédié à l'intelligence artificielle développé avec Python et FastAPI, un backend d'intégration développé avec Spring Boot et intégrant des notifications en temps réel, ainsi qu'un tableau de bord web développé avec Angular 19.

La solution vise ainsi à proposer une supervision intelligente capable d'exploiter les données de télémétrie, d'identifier des comportements inhabituels et de fournir aux opérateurs des informations exploitables pour faciliter la détection et le suivi des anomalies.

Mots-clés : IoT, fuite d'eau, intelligence artificielle, Isolation Forest, FastAPI, Spring Boot, Angular, Minimum Night Flow, détection d'anomalies.

Abstract

This report presents the design and implementation of **Droppy**, a multi-service software suite dedicated to automatic water leak and anomaly detection in distribution networks.

Developed at SFM Connect, the solution combines data collected from IoT sensors, hydraulic business rules such as the *Minimum Night Flow* and valve-flow inconsistency detection, together with machine learning techniques including **Isolation Forest** and a consumption forecasting model.

The architecture is based on three main components : an artificial intelligence micro-service developed with Python and FastAPI, an integration backend developed with Spring Boot and supporting real-time notifications, and a web dashboard developed with Angular 19.

The proposed solution aims to provide intelligent supervision capable of processing telemetry data, identifying unusual behaviors, and providing operators with actionable information to facilitate anomaly detection and monitoring.

Keywords : IoT, water leak, artificial intelligence, Isolation Forest, FastAPI, Spring Boot, Angular, Minimum Night Flow, anomaly detection.

Table des matières

Remerciements	2
Résumé	3
Abstract	4
Liste des figures	9
Liste des tableaux	10
1 Introduction générale	11
1.1 Contexte général	11
1.2 Présentation de l'entreprise et du projet	12
1.3 Présentation de la solution Droppy	12
1.4 Problématique	13
1.5 Approche proposée	14
1.6 Architecture générale de la solution	14
1.7 Objectifs du projet	15
1.8 Méthodologie et organisation du projet	16
1.9 Structure du rapport	17
Conclusion du chapitre	17
2 Analyse des besoins et cahier des charges	19
2.1 Introduction	19
2.2 Compréhension du domaine métier	19
2.2.1 Enjeux de la gestion des réseaux d'eau	19
2.2.2 Typologie des anomalies	20
2.2.3 Données disponibles et télémétrie IoT	20
2.2.4 Enjeux de la détection précoce	21
2.3 Étude de l'existant	21
2.3.1 Surveillance traditionnelle	21

2.3.2	Approches basées sur les seuils	22
2.3.3	Limites identifiées	22
2.3.4	Besoin d'une supervision intelligente	22
2.4	Vision fonctionnelle de Droppy	23
2.4.1	Vision globale	23
2.4.2	Chaîne globale de traitement	23
2.4.3	Valeur ajoutée de la solution	24
2.4.4	Résultats attendus	24
2.5	Identification des acteurs	25
2.5.1	Opérateur / gestionnaire	25
2.5.2	Capteurs et équipements IoT	25
2.5.3	Backend applicatif	26
2.5.4	Service d'intelligence artificielle	26
2.5.5	Interface de supervision	27
2.6	Besoins fonctionnels	27
2.6.1	Gestion des équipements	27
2.6.2	Ingestion des télémetries	27
2.6.3	Nettoyage et prétraitement	27
2.6.4	Détection des anomalies	28
2.6.5	Détection des fuites	28
2.6.6	Prévision de consommation	28
2.6.7	Gestion des alertes	28
2.6.8	Supervision en temps réel	28
2.6.9	Historisation	29
2.6.10	Réentraînement des modèles	29
2.7	Besoins non fonctionnels	29
2.7.1	Performance	29
2.7.2	Fiabilité	29
2.7.3	Temps réel	30
2.7.4	Sécurité	30
2.7.5	Maintenabilité	30
2.7.6	Évolutivité	30
2.7.7	Ergonomie	30
2.8	Contraintes du projet	31
2.8.1	Hétérogénéité technologique	31
2.8.2	Nature temporelle des données	31
2.8.3	Communication inter-services	31

2.8.4	Gestion de la persistance	31
2.8.5	Validation de la solution	32
2.9	Cahier des charges fonctionnel	32
2.9.1	Tableau des exigences	33
2.9.2	Priorisation des exigences	33
2.10	Critères de réussite	34
2.10.1	Critères fonctionnels	34
2.10.2	Critères techniques	34
2.10.3	Critères liés à l'utilisateur	35
2.11	Conclusion	35
3	Conception et architecture de la solution Droppy	36
3.1	Introduction	36
3.2	Architecture globale	36
3.3	Modélisation fonctionnelle	37
3.4	Modélisation dynamique	38
3.4.1	Détection d'une anomalie	39
3.4.2	Prévision de consommation	39
3.5	Modélisation statique	40
3.6	Conclusion	42
4	Réalisation et présentation de l'application Droppy	43
4.1	Introduction	43
4.2	Présentation générale de l'application	43
4.2.1	Écran principal	43
4.2.2	Organisation de l'interface	44
4.3	Visualisation des données	44
4.3.1	Graphiques de suivi	44
4.3.2	Visualisation cartographique	45
4.4	Analyse intelligente	45
4.4.1	Déclenchement d'une analyse	46
4.5	Conclusion	46
5	Intelligence artificielle et détection des anomalies	47
5.1	Introduction	47
5.2	Pipeline de traitement des données	47
5.3	Stratégie de détection des anomalies	48

5.4	Détection basée sur les règles métier	49
5.4.1	Minimum Night Flow	49
5.4.2	Incohérence liée aux vannes	49
5.5	Détection par Isolation Forest	50
5.6	Agrégation des résultats	51
5.7	Résultat de l'analyse	51
5.8	Exposition des fonctionnalités IA	51
5.9	Synthèse	52
5.10	Conclusion	52
6	Conclusion générale et perspectives	53
6.1	Conclusion générale	53
6.2	Apports du projet	54
6.3	Limites de la solution	54
6.4	Perspectives	54
6.4.1	Amélioration des modèles d'intelligence artificielle	55
6.4.2	Extension de la supervision	55
6.4.3	Amélioration du système d'alertes	55
6.4.4	Déploiement dans un environnement Cloud	55
6.4.5	Développement d'une application mobile	55
6.4.6	Évolution vers une maintenance prédictive	55
6.5	Conclusion finale	55
	Bibliographie	57
A	Annexes	58
A.1	User Stories implémentées	58
A.2	Configuration de déploiement	58
A.3	Structure du projet source	58

Table des figures

3.1	Architecture globale de la solution Droppy	37
3.2	Diagramme de cas d'utilisation de Droppy	38
3.3	Séquence de détection d'une anomalie	39
3.4	Séquence de prévision de consommation	40
3.5	Diagramme de classes principal de Droppy	41
4.1	Écran principal du dashboard Droppy	44
4.2	Organisation générale de l'interface	44
4.3	Visualisation graphique des données	45
4.4	Visualisation cartographique des anomalies	45
4.5	Déclenchement d'une analyse	46
5.1	Pipeline de traitement des données dans Droppy	48
5.2	Stratégie hybride de détection des anomalies	49
5.3	Principe d'utilisation de l'Isolation Forest dans Droppy	50

Liste des tableaux

2.1	Synthèse des exigences fonctionnelles de Droppy	33
A.1	Correspondance User Stories / modules	58
A.2	Services et ports	58

Introduction générale

1.1 Contexte général

L'eau constitue une ressource naturelle essentielle au développement des sociétés et au fonctionnement de nombreuses activités humaines. Sa gestion efficace représente aujourd'hui un enjeu majeur, particulièrement dans les régions confrontées à des périodes de stress hydrique et à une augmentation constante des besoins en consommation. Dans ce contexte, la réduction des pertes d'eau et l'amélioration de la surveillance des installations hydrauliques constituent des problématiques importantes pour les entreprises, les collectivités et les gestionnaires d'infrastructures.

Les réseaux de distribution et les installations hydrauliques peuvent être affectés par différents types de dysfonctionnements, notamment les fuites, les consommations inhabituelles, les variations anormales de débit ou encore les problèmes liés au fonctionnement des vannes. Lorsque ces événements ne sont pas détectés rapidement, ils peuvent entraîner une consommation excessive d'eau, une augmentation des coûts d'exploitation, des dégradations matérielles et, dans certains cas, des conséquences environnementales importantes.

Les méthodes traditionnelles de surveillance reposent généralement sur des contrôles périodiques et des inspections manuelles. Bien que ces approches permettent d'identifier certains problèmes, elles présentent des limites lorsqu'il s'agit de détecter rapidement des anomalies apparaissant entre deux inspections. L'exploitation continue des données issues de capteurs connectés constitue ainsi une approche permettant d'améliorer la surveillance des installations et de réduire le délai de détection des événements anormaux.

L'intégration de l'Internet des Objets (*Internet of Things* – *IoT*) permet justement de collecter en continu des données provenant des équipements physiques. Ces données peuvent ensuite être exploitées par des systèmes logiciels capables de les analyser automatiquement et d'assister les opérateurs dans la supervision des installations.

1.2 Présentation de l'entreprise et du projet

C'est dans ce contexte que s'inscrit le présent projet, réalisé au sein de **SFM Connect**, branche spécialisée dans les solutions liées à l'Internet des Objets (*Internet of Things* – *IoT*) du **Groupe SFM**, à Tunis.

SFM Connect développe des solutions permettant de connecter des équipements physiques à des plateformes numériques afin de collecter, traiter et exploiter leurs données. Dans le domaine de la gestion de l'eau, le projet **Droppy** s'inscrit dans cette démarche en proposant une solution destinée à améliorer la supervision des installations hydrauliques à partir des données provenant de capteurs IoT.

Ces capteurs transmettent régulièrement différentes informations relatives au fonctionnement du réseau, notamment le débit d'eau (**flow_rate**), la consommation cumulative (**consumption**) ainsi que l'état d'une vanne (**state**), généralement représenté par les états **ON** et **OFF**. Chaque équipement est identifié de manière unique, notamment à travers son adresse MAC, permettant ainsi d'associer les différentes mesures au capteur correspondant.

L'exploitation de ces données permet de passer d'une simple collecte d'informations à une véritable approche de supervision intelligente. Le système développé dans le cadre de ce projet vise ainsi à analyser automatiquement les données de télémétrie afin d'identifier des comportements inhabituels pouvant correspondre à des fuites ou à d'autres anomalies de fonctionnement.

1.3 Présentation de la solution Droppy

Le projet **Droppy – Water Leak Detection & Anomaly Analysis Suite** consiste en la conception et la réalisation d'une plateforme logicielle intelligente dédiée à la détection des fuites d'eau et à l'analyse des anomalies provenant des capteurs IoT.

La solution adopte une architecture distribuée organisée autour de plusieurs composants spécialisés. Elle associe une interface web développée avec **Angular 19**, un backend principal développé avec **Spring Boot 3** et un service dédié aux traitements d'intelligence artificielle développé avec **FastAPI** et Python.

Cette organisation permet de séparer les différentes responsabilités du système. Le backend constitue le service central de la plateforme et assure notamment l'intégration des données, l'orchestration des traitements, la gestion de la persistance ainsi que la communication avec l'interface utilisateur. Le service d'intelligence artificielle prend

en charge les traitements nécessaires à l'analyse des données et à la détection des comportements inhabituels.

Le tableau de bord constitue l'interface principale de supervision. Il permet aux utilisateurs de consulter les informations relatives aux capteurs, d'analyser les données historiques, de visualiser les anomalies détectées et de recevoir des notifications lorsqu'un événement important est identifié.

Les choix techniques ainsi que l'organisation détaillée de cette architecture seront présentés dans les chapitres consacrés à l'analyse des besoins et à la conception de la solution.

1.4 Problématique

La principale difficulté d'un système de surveillance des installations hydrauliques réside dans la capacité à distinguer un comportement normal d'un comportement réellement anormal. Une augmentation du débit peut, par exemple, correspondre à une utilisation normale de l'installation, mais elle peut également être le signe d'une fuite. De la même manière, un état particulier de la vanne associé à un débit inattendu peut révéler une incohérence dans le fonctionnement du système.

L'analyse d'un seul indicateur ne permet donc pas toujours d'obtenir une détection suffisamment fiable. Il devient nécessaire de prendre en considération plusieurs signaux provenant des capteurs et d'étudier leur évolution dans le temps.

Par ailleurs, les règles statiques peuvent être efficaces pour identifier certains cas connus, mais elles présentent des limites face à des comportements inhabituels qui n'ont pas été explicitement définis à l'avance. L'utilisation de techniques d'intelligence artificielle et d'apprentissage automatique permet alors de compléter les règles métier en recherchant automatiquement des comportements atypiques dans les données.

La problématique principale de ce projet peut ainsi être formulée comme suit :

Comment concevoir et mettre en œuvre une plateforme intelligente, fiable et évolutive permettant d'exploiter les données issues de capteurs IoT afin de détecter automatiquement les fuites d'eau et les anomalies de fonctionnement, tout en fournissant aux opérateurs une interface de supervision capable de présenter les résultats et de générer des alertes en temps réel ?

1.5 Approche proposée

Afin de répondre à cette problématique, le projet adopte une approche hybride combinant des méthodes déterministes et des techniques d'apprentissage automatique.

Dans un premier temps, les données provenant des capteurs sont nettoyées et prétraitées afin de traiter les valeurs incohérentes et de préparer les données pour les différentes étapes d'analyse. Une attention particulière est accordée à l'exploitation des mesures de débit et de consommation en tant que séries temporelles.

Dans un deuxième temps, des règles métier sont appliquées afin d'identifier certaines situations particulières. Parmi ces mécanismes figure notamment l'analyse du *Minimum Night Flow* (MNF), permettant d'étudier les consommations observées pendant les périodes nocturnes, ainsi que la détection d'incohérences entre l'état d'une vanne et les mesures de débit.

En complément de ces règles, le système intègre des méthodes d'apprentissage automatique. L'algorithme **Isolation Forest**, utilisé dans une approche de détection non supervisée, permet d'identifier des observations présentant un comportement inhabituel par rapport aux données observées.

Un mécanisme de prévision de la consommation est également utilisé afin de comparer les tendances attendues aux comportements réellement observés.

Cette combinaison permet ainsi de dépasser les limites d'une détection reposant uniquement sur des seuils fixes. Les résultats des différents mécanismes d'analyse peuvent ensuite être exploités par le système de supervision afin d'informer les opérateurs et de faciliter l'identification des situations nécessitant une intervention.

1.6 Architecture générale de la solution

L'architecture générale de Droppy repose sur trois composants principaux qui collaborent pour assurer la collecte, le traitement, l'analyse et la visualisation des données.

Le premier composant correspond au **frontend Angular 19**. Il constitue l'interface utilisateur de la plateforme et permet notamment d'afficher les informations relatives aux capteurs, les anomalies détectées, les données historiques ainsi que différentes visualisations graphiques. Il permet également de recevoir des événements en temps réel.

Le deuxième composant est le **backend Spring Boot 3**, qui constitue le service central

de l'application. Il assure l'intégration des différents traitements, la gestion des données, la persistance des informations et la communication avec l'interface utilisateur.

Le troisième composant correspond au **service d'intelligence artificielle FastAPI**. Il regroupe les traitements liés à la préparation des données, à la détection des anomalies et à la prévision. Il communique avec le backend à travers des API afin de fournir les résultats nécessaires à la plateforme.

Cette organisation permet de séparer clairement les responsabilités entre l'interface utilisateur, la logique applicative et les traitements d'intelligence artificielle. Elle facilite ainsi la maintenance du système et permet d'envisager son évolution vers de nouveaux types de capteurs, de nouvelles méthodes d'analyse ou de nouvelles fonctionnalités.

L'architecture détaillée, les flux de communication entre les composants ainsi que les choix technologiques seront présentés dans les chapitres consacrés à la conception de la solution.

1.7 Objectifs du projet

L'objectif général de ce projet est de concevoir et développer une plateforme intelligente permettant d'améliorer la détection des fuites d'eau et la supervision des installations hydrauliques à partir de données IoT.

Plus précisément, les objectifs poursuivis sont les suivants :

- ▶ Mettre en place un pipeline permettant de charger, nettoyer et prétraiter les données de télémétrie issues des capteurs IoT ;
- ▶ Développer des mécanismes de détection basés sur des règles métier afin d'identifier les comportements anormaux liés notamment au débit, à la consommation et à l'état des vannes ;
- ▶ Intégrer des méthodes d'apprentissage automatique, notamment l'algorithme **Isolation Forest**, afin de détecter automatiquement des comportements atypiques ;
- ▶ Mettre en place un mécanisme de prévision de la consommation permettant d'analyser les tendances et d'identifier d'éventuels écarts par rapport au comportement attendu ;
- ▶ Développer un service d'intelligence artificielle avec **FastAPI** afin d'exposer les fonctionnalités d'analyse, de prédiction et de prévision à travers des API

REST ;

- ▶ Développer un backend central avec **Spring Boot 3** permettant d'assurer l'intégration des différents composants de la plateforme ;
- ▶ Mettre en œuvre un système de notifications en temps réel afin d'informer rapidement les opérateurs lorsqu'une anomalie importante est détectée ;
- ▶ Concevoir un tableau de bord interactif avec **Angular 19** permettant aux utilisateurs de visualiser les données, les anomalies et les informations relatives aux capteurs ;
- ▶ Assurer la persistance et la gestion structurée des données afin de permettre la consultation et l'analyse des historiques ;
- ▶ Mettre en place des tests permettant de vérifier le bon fonctionnement des différents composants de la solution.

1.8 Méthodologie et organisation du projet

La réalisation du projet a été organisée autour de plusieurs étapes successives permettant de progresser progressivement de l'analyse du besoin jusqu'à la validation de la solution.

La première étape a consisté à comprendre le fonctionnement du système IoT et la structure des données provenant des capteurs. Cette phase a permis d'identifier les différentes variables disponibles et de déterminer les informations nécessaires à la détection des anomalies.

La deuxième étape a concerné le traitement et la préparation des données. Les données brutes ont été analysées, nettoyées et transformées afin de constituer une base exploitable pour les différents mécanismes de détection.

La troisième étape a consisté à concevoir et développer les mécanismes d'analyse en combinant les règles métier et les méthodes d'apprentissage automatique. Cette phase a notamment permis d'intégrer la détection des comportements atypiques ainsi que les mécanismes de prévision de la consommation.

La quatrième étape a été consacrée au développement de l'architecture applicative. Les différents services ont été mis en place afin d'assurer la communication entre les composants de la plateforme. En parallèle, l'interface de supervision a été développée afin de permettre la visualisation et l'exploitation des résultats.

Enfin, une phase de tests et de validation a permis de vérifier le comportement des

différentes composantes du système et d'évaluer la cohérence des résultats produits.

1.9 Structure du rapport

Afin de présenter de manière progressive les différentes étapes de réalisation du projet, ce rapport est organisé en plusieurs chapitres complémentaires.

Le premier chapitre présente le contexte général du projet, l'entreprise d'accueil, la solution Droppy, la problématique ainsi que les objectifs poursuivis.

Le deuxième chapitre est consacré à l'analyse des besoins et au cahier des charges. Il présente notamment les acteurs du système, les besoins fonctionnels et non fonctionnels, les contraintes ainsi que les critères permettant d'évaluer la réussite de la solution.

Les chapitres suivants présentent progressivement l'analyse de l'existant, la conception de la solution, l'architecture logicielle, les modèles UML ainsi que les choix technologiques adoptés.

Une partie du rapport est ensuite consacrée à la réalisation de la plateforme. Elle présente les principales fonctionnalités développées au niveau du backend, du service d'intelligence artificielle et de l'interface de supervision.

Les mécanismes d'intelligence artificielle et de détection des anomalies sont également détaillés, notamment le prétraitement des données, les règles métier, l'algorithme **Isolation Forest** ainsi que les mécanismes de prévision.

Enfin, le rapport présente la stratégie de tests et de validation mise en place afin d'évaluer le fonctionnement de la solution, avant de terminer par une conclusion générale présentant les principales contributions réalisées, les limites identifiées et les perspectives d'évolution de la plateforme.

Conclusion du chapitre

Ce premier chapitre a permis de présenter le contexte général dans lequel s'inscrit le projet **Droppy**, ainsi que les enjeux liés à la surveillance intelligente des installations hydrauliques. L'analyse du problème a mis en évidence les limites des approches traditionnelles et la nécessité de disposer d'une solution capable d'exploiter en continu les données issues des capteurs IoT.

La solution proposée repose sur une approche combinant traitement des données, règles métier et techniques d'apprentissage automatique afin d'améliorer la détection des

comportements anormaux. L'architecture retenue permet également de séparer les différentes responsabilités du système et de fournir une interface de supervision adaptée aux besoins des utilisateurs.

Après avoir défini le contexte, la problématique et les objectifs du projet, le chapitre suivant sera consacré à l'analyse détaillée des besoins et à l'élaboration du cahier des charges. Cette étape permettra de transformer les objectifs identifiés en exigences fonctionnelles et non fonctionnelles précises, constituant ainsi la base de la conception de la solution.

Analyse des besoins et cahier des charges

2.1 Introduction

Après avoir présenté dans le chapitre précédent le contexte général du projet Droppy, sa problématique ainsi que les objectifs poursuivis, ce chapitre est consacré à l'analyse des besoins et à la définition du cahier des charges de la solution.

L'objectif de cette phase est de traduire le problème métier en exigences concrètes permettant de guider la conception et la réalisation de la plateforme. Il s'agit notamment d'identifier les utilisateurs concernés, les sources de données, les fonctionnalités attendues ainsi que les contraintes techniques et qualitatives auxquelles le système doit répondre.

L'analyse commence par une étude du domaine de la gestion des réseaux d'eau et des problématiques liées aux fuites et aux comportements anormaux. Une étude de l'existant permet ensuite de mettre en évidence les limites des approches traditionnelles et de justifier la nécessité d'une supervision plus automatisée et intelligente.

La vision fonctionnelle de Droppy est ensuite présentée afin de décrire la chaîne globale allant de la collecte des données jusqu'à leur analyse et à leur restitution à l'opérateur.

Enfin, les acteurs du système, les besoins fonctionnels, les besoins non fonctionnels, les contraintes du projet ainsi que les critères de réussite sont définis afin de constituer le cahier des charges de la solution.

2.2 Compréhension du domaine métier

2.2.1 Enjeux de la gestion des réseaux d'eau

La gestion efficace d'un réseau de distribution d'eau nécessite une surveillance régulière des différents équipements et des flux de consommation. Le gestionnaire doit être capable

de suivre l'évolution des débits, des consommations ainsi que l'état des équipements afin d'identifier rapidement toute situation inhabituelle.

Dans un réseau instrumenté, les capteurs permettent de transformer les équipements physiques en sources de données exploitables par une plateforme logicielle. La surveillance ne repose alors plus uniquement sur des inspections ponctuelles, mais peut s'appuyer sur une observation continue du comportement du réseau.

Cette évolution présente un intérêt particulier dans le contexte de la détection des fuites. Une fuite non détectée peut entraîner une perte importante d'eau et provoquer des coûts supplémentaires pour l'exploitant. La rapidité de détection constitue donc un élément essentiel de la supervision.

2.2.2 Typologie des anomalies

Dans le cadre du projet Droppy, une anomalie correspond à un comportement qui s'écarte du fonctionnement habituellement observé.

Plusieurs situations peuvent notamment être considérées :

- ▶ une augmentation inhabituelle du débit ;
- ▶ une consommation anormalement élevée pendant une période de faible activité ;
- ▶ une consommation nocturne inhabituelle ;
- ▶ une incohérence entre l'état d'une vanne et le débit observé ;
- ▶ une évolution inhabituelle d'une série temporelle ;
- ▶ un comportement globalement atypique détecté automatiquement par un modèle d'apprentissage automatique.

Il est important de souligner qu'une anomalie ne correspond pas nécessairement à une fuite. Elle constitue plutôt un signal nécessitant une analyse complémentaire. La plateforme doit donc fournir suffisamment d'informations pour permettre à l'opérateur d'interpréter le comportement détecté.

2.2.3 Données disponibles et télémétrie IoT

Les capteurs IoT constituent la principale source de données de Droppy.

Les informations exploitées peuvent notamment comprendre :

- ▶ le débit d'eau ;
- ▶ la consommation ;
- ▶ l'état d'une vanne ;
- ▶ l'identifiant du dispositif ;
- ▶ l'horodatage associé à la mesure.

Chaque équipement doit pouvoir être identifié de manière unique afin de maintenir la correspondance entre les mesures reçues et leur source.

La dimension temporelle des données constitue également un élément important. L'analyse d'une mesure isolée étant généralement insuffisante, le système doit pouvoir exploiter l'évolution des données au cours du temps.

2.2.4 Enjeux de la détection précoce

La détection précoce représente un enjeu majeur pour la surveillance des réseaux d'eau.

Une anomalie détectée rapidement permet à l'opérateur d'analyser la situation et, lorsque cela est nécessaire, de déclencher une intervention avant que le problème ne provoque des pertes importantes.

L'objectif de Droppy est donc de réduire autant que possible le délai entre l'apparition d'un comportement inhabituel et son identification par l'utilisateur.

Cette exigence justifie la mise en place d'une chaîne automatisée combinant collecte des données, analyse, détection et notification.

2.3 Étude de l'existant

2.3.1 Surveillance traditionnelle

Les approches traditionnelles de surveillance des réseaux d'eau reposent principalement sur les inspections périodiques, les relevés manuels et l'utilisation de seuils définis à l'avance.

Dans ce contexte, l'opérateur consulte les mesures disponibles et recherche des valeurs pouvant indiquer un dysfonctionnement.

Cette approche reste pertinente pour certains scénarios, mais elle devient plus difficile à

maintenir lorsque le nombre d'équipements et la fréquence des mesures augmentent.

2.3.2 Approches basées sur les seuils

Une méthode courante consiste à définir un seuil à partir duquel une alerte est déclenchée.

Par exemple, une règle peut considérer qu'un débit dépassant une certaine valeur constitue une situation potentiellement anormale.

Cependant, le comportement normal d'un équipement peut varier selon l'heure, le jour, la période d'utilisation ou les caractéristiques du site. Un seuil fixe ne permet donc pas toujours de représenter correctement l'ensemble des situations possibles.

2.3.3 Limites identifiées

Plusieurs limites peuvent ainsi être identifiées.

Premièrement, les inspections manuelles ne garantissent pas une détection immédiate des anomalies.

Deuxièmement, les seuils statiques peuvent générer des faux positifs lorsque la variation observée correspond à un fonctionnement normal.

Troisièmement, certaines anomalies peuvent rester invisibles lorsqu'elles ne dépassent pas les seuils prédéfinis.

Enfin, l'analyse manuelle devient difficile lorsque plusieurs capteurs produisent continuellement des données.

Ces limites montrent qu'une approche automatisée est nécessaire afin d'exploiter efficacement les volumes de données disponibles.

2.3.4 Besoin d'une supervision intelligente

Une supervision intelligente doit être capable de combiner plusieurs sources d'information et plusieurs mécanismes d'analyse.

Dans Droppy, cette approche repose sur une combinaison entre des règles métier et des techniques d'apprentissage automatique.

Les règles métier permettent d'identifier des situations connues, tandis que les modèles d'apprentissage automatique permettent de rechercher des comportements atypiques dans les données.

Cette complémentarité permet d'obtenir une analyse plus riche qu'une approche reposant

uniquement sur des seuils fixes.

2.4 Vision fonctionnelle de Droppy

2.4.1 Vision globale

Droppy est conçu comme une plateforme de supervision intelligente permettant de transformer les données brutes provenant des équipements IoT en informations exploitables par les opérateurs.

La solution est organisée autour de trois composants logiciels principaux :

- ▶ **Angular 19**, utilisé pour développer le tableau de bord de supervision ;
- ▶ **Spring Boot 3**, utilisé comme service backend central pour l'intégration, l'orchestration et la gestion des événements ;
- ▶ **FastAPI et Python**, utilisés pour les traitements de données et les mécanismes d'intelligence artificielle.

Le dépôt du projet confirme cette organisation en distinguant notamment le frontend **dashboard-angular**, le service backend **core-service** et le moteur d'intelligence artificielle situé dans le répertoire **src**. :contentReference[oaicite :3]index=3

2.4.2 Chaîne globale de traitement

Le fonctionnement fonctionnel de Droppy peut être représenté comme une chaîne composée des étapes suivantes :

1. collecte des données provenant des équipements IoT ;
2. ingestion des données par le système ;
3. nettoyage et préparation des données ;
4. application des règles métier ;
5. analyse par les modèles d'apprentissage automatique ;
6. génération des résultats de détection ou de prévision ;

7. enregistrement des informations importantes ;
8. transmission des événements au tableau de bord ;
9. consultation et interprétation des résultats par l'opérateur.

Cette chaîne permet de transformer progressivement une donnée brute en une information pouvant être utilisée pour la supervision et la prise de décision.

2.4.3 Valeur ajoutée de la solution

La valeur ajoutée de Droppy réside principalement dans l'intégration de plusieurs mécanismes au sein d'une même plateforme.

La solution ne se limite pas à afficher les mesures provenant des capteurs. Elle permet également de les nettoyer, de les analyser et d'identifier automatiquement certains comportements inhabituels.

L'utilisation d'un modèle tel que **Isolation Forest** permet notamment de rechercher des observations atypiques sans dépendre exclusivement de seuils prédéfinis.

La plateforme intègre également des mécanismes de prévision ainsi que des notifications permettant de transmettre rapidement les événements importants à l'utilisateur.

Le dépôt du projet indique notamment l'existence des fonctionnalités de réentraînement des modèles, de prédiction combinée et de prévision de consommation. :contentReference[oaicite :4]index=4

2.4.4 Résultats attendus

À partir des données collectées, le système doit permettre d'obtenir plusieurs types de résultats :

- ▶ identification de comportements atypiques ;
- ▶ détection de situations pouvant correspondre à des fuites ;
- ▶ production de prévisions de consommation ;
- ▶ génération d'événements d'alerte ;
- ▶ visualisation des résultats dans le tableau de bord ;
- ▶ conservation des informations nécessaires à l'analyse historique.

L'objectif final est de fournir à l'opérateur une information synthétique et exploitable plutôt qu'un simple ensemble de mesures brutes.

2.5 Identification des acteurs

L'identification des acteurs permet de déterminer les différentes entités participant au fonctionnement de la solution et leurs responsabilités.

Contrairement à une application classique, Droppy implique à la fois des utilisateurs humains, des équipements physiques et des services logiciels.

2.5.1 Opérateur / gestionnaire

L'opérateur constitue l'utilisateur principal de la plateforme.

Il utilise le tableau de bord pour :

- ▶ consulter les équipements ;
- ▶ visualiser les données de télémétrie ;
- ▶ suivre les anomalies détectées ;
- ▶ consulter les historiques ;
- ▶ recevoir les notifications ;
- ▶ analyser les tendances de consommation.

Son rôle est principalement orienté vers la supervision et l'interprétation des résultats fournis par le système.

2.5.2 Capteurs et équipements IoT

Les équipements IoT représentent les sources physiques des données.

Ils transmettent les mesures nécessaires au fonctionnement de la chaîne d'analyse. Ces données peuvent notamment concerner le débit, la consommation et l'état des vannes.

Les équipements doivent être identifiables afin de permettre au système d'associer chaque mesure au dispositif correspondant.

2.5.3 Backend applicatif

Le backend Spring Boot joue le rôle de service central d'intégration.

Il assure notamment :

- ▶ la réception et l'intégration des données ;
- ▶ l'orchestration des traitements ;
- ▶ la communication avec le service d'intelligence artificielle ;
- ▶ la persistance des informations ;
- ▶ le routage des événements vers le tableau de bord.

Le dépôt indique notamment l'utilisation de Spring Boot 3, de mécanismes de planification et de WebSockets pour les notifications en temps réel. :contentReference[oaicite :5]index=5

2.5.4 Service d'intelligence artificielle

Le microservice FastAPI constitue le moteur d'analyse de Droppy.

Il prend en charge les traitements liés aux données et à l'intelligence artificielle, notamment :

- ▶ le nettoyage des données ;
- ▶ le prétraitement ;
- ▶ l'application de certaines règles métier ;
- ▶ la détection des anomalies ;
- ▶ la prévision de consommation ;
- ▶ le réentraînement des modèles.

Le projet expose notamment des opérations de réentraînement, de prédiction et de prévision pour un équipement donné. :contentReference[oaicite :6]index=6

2.5.5 Interface de supervision

L'interface Angular constitue le point d'accès principal de l'opérateur.

Elle permet de centraliser les résultats produits par les différents services et de les présenter sous une forme visuelle.

Le tableau de bord permet notamment d'afficher les anomalies, les données historiques et les graphiques de télémétrie tout en recevant les notifications en temps réel. :conten-
tReference[oaicite :7]index=7

2.6 Besoins fonctionnels

Les besoins fonctionnels définissent les services que la plateforme doit fournir pour répondre aux objectifs identifiés.

2.6.1 Gestion des équipements

BF01 – Gestion des équipements

Le système doit permettre d'identifier et de consulter les équipements surveillés.

Chaque équipement doit pouvoir être associé à ses différentes mesures et à son historique afin de faciliter le suivi individuel.

2.6.2 Ingestion des télémétries

BF02 – Collecte des données

Le système doit permettre de recevoir et d'intégrer les données produites par les équipements IoT.

Les données doivent être correctement associées à leur équipement et à leur instant de mesure.

2.6.3 Nettoyage et prétraitement

BF03 – Prétraitement

Le système doit être capable de nettoyer et de préparer les données avant leur analyse.

Cette étape doit notamment permettre de traiter les valeurs manquantes, les incohérences

et les transformations nécessaires à l'exploitation des séries temporelles.

2.6.4 Détection des anomalies

BF04 – Détection des anomalies

La plateforme doit identifier automatiquement les comportements atypiques présents dans les données de télémétrie.

Un modèle de type **Isolation Forest** est utilisé pour contribuer à cette détection.

2.6.5 Détection des fuites

BF05 – Détection des fuites

Le système doit permettre d'identifier les situations pouvant correspondre à une fuite.

La détection peut s'appuyer sur plusieurs indicateurs tels que le débit, la consommation, l'état des vannes et les comportements observés pendant les périodes de faible activité.

2.6.6 Préviation de consommation

BF06 – Préviation

La plateforme doit permettre de produire des prévisions de consommation afin de comparer les valeurs observées avec les tendances attendues.

Cette fonctionnalité peut également contribuer à identifier des écarts significatifs dans le comportement d'un équipement.

2.6.7 Gestion des alertes

BF07 – Alertes

Lorsqu'une anomalie importante est identifiée, le système doit pouvoir générer un événement destiné à l'opérateur.

Les événements critiques doivent pouvoir être transmis rapidement au tableau de bord.

2.6.8 Supervision en temps réel

BF08 – Supervision

Le système doit fournir une interface permettant à l'opérateur de suivre les informations

importantes relatives au réseau.

La plateforme doit notamment permettre de consulter les anomalies, les graphiques, les équipements et les notifications.

2.6.9 Historisation

BF09 – Historique

Le système doit conserver les informations nécessaires au suivi historique des données et des anomalies.

L'historique permet notamment de comparer le comportement actuel d'un équipement avec ses observations précédentes.

2.6.10 Réentraînement des modèles

BF10 – Gestion des modèles

Le système doit permettre de réentraîner les modèles d'intelligence artificielle pour un équipement donné lorsque cela est nécessaire.

Cette fonctionnalité permet d'adapter les modèles aux nouvelles données disponibles.

2.7 Besoins non fonctionnels

2.7.1 Performance

BNF01 – Performance

La plateforme doit traiter les données avec un temps de réponse acceptable afin de permettre une utilisation fluide du tableau de bord.

2.7.2 Fiabilité

BNF02 – Fiabilité

Les données doivent être traitées de manière cohérente et les mécanismes d'analyse doivent limiter les erreurs liées aux données incomplètes ou incohérentes.

2.7.3 Temps réel

BNF03 – Temps réel

Les événements importants doivent pouvoir être transmis rapidement à l'interface de supervision.

L'utilisation des WebSockets permet au backend de transmettre des notifications sans nécessiter une actualisation permanente de la page.

2.7.4 Sécurité

BNF04 – Sécurité

Les communications entre les différents composants doivent être protégées et les accès aux fonctionnalités sensibles doivent être contrôlés.

2.7.5 Maintenabilité

BNF05 – Maintenabilité

La séparation entre frontend, backend et service d'intelligence artificielle doit faciliter la maintenance et l'évolution du système.

2.7.6 Évolutivité

BNF06 – Évolutivité

La solution doit pouvoir intégrer de nouveaux équipements, de nouvelles méthodes d'analyse et de nouvelles fonctionnalités sans nécessiter une refonte complète de l'architecture.

2.7.7 Ergonomie

BNF07 – Ergonomie

Le tableau de bord doit présenter les informations de manière claire et compréhensible afin de permettre à l'opérateur d'identifier rapidement les situations nécessitant son attention.

2.8 Contraintes du projet

La réalisation de Droppy doit respecter plusieurs contraintes techniques et organisationnelles.

2.8.1 Hétérogénéité technologique

La solution repose sur plusieurs technologies et environnements :

- ▶ Python et FastAPI pour le service d'intelligence artificielle ;
- ▶ Java et Spring Boot pour le backend ;
- ▶ TypeScript et Angular 19 pour le frontend ;
- ▶ outils de gestion de données et de modèles pour le pipeline IA.

Cette diversité nécessite une définition claire des interfaces entre les différents composants.

2.8.2 Nature temporelle des données

Les données de télémétrie sont associées à des instants de mesure. Leur ordre temporel doit donc être conservé afin de permettre l'analyse des évolutions et des tendances.

2.8.3 Communication inter-services

Le backend doit assurer une communication fiable avec le service d'intelligence artificielle et avec le frontend.

Le dépôt du projet indique notamment l'utilisation de HTTP/REST entre les services ainsi que de WebSockets pour les notifications temps réel. :contentReference[oaicite :8]index=8

2.8.4 Gestion de la persistance

Les informations nécessaires au suivi des équipements, des événements et des analyses doivent être correctement persistées.

Le projet utilise notamment Liquibase pour la gestion et le versionnement des évolutions

du schéma de base de données. :contentReference[oaicite :9]index=9

2.8.5 Validation de la solution

La solution doit être accompagnée de tests permettant de vérifier les différents composants.

Le dépôt comprend notamment des tests dédiés au pipeline Python et aux fonctionnalités de l'API, ainsi que des procédures de test pour les différents composants de l'application. :contentReference[oaicite :10]index=10

2.9 Cahier des charges fonctionnel

Le cahier des charges fonctionnel synthétise les principales exigences identifiées au cours de l'analyse.

2.9.1 Tableau des exigences

Table 2.1. Synthèse des exigences fonctionnelles de Droppy

ID	Fonctionnalité	Description	Priorité
BF01	Gestion des équipements	Identifier et consulter les équipements surveillés.	Haute
BF02	Collecte des données	Recevoir et intégrer les télémetries provenant des capteurs.	Haute
BF03	Prétraitement	Nettoyer et préparer les données avant leur analyse.	Haute
BF04	Détection des anomalies	Identifier automatiquement les comportements atypiques.	Haute
BF05	Détection des fuites	Identifier les situations pouvant correspondre à une fuite.	Haute
BF06	Prévision	Analyser les tendances et prévoir la consommation.	Moyenne
BF07	Alertes	Informier l'opérateur lorsqu'une anomalie importante est détectée.	Haute
BF08	Supervision	Visualiser les données, équipements et anomalies.	Haute
BF09	Historique	Permettre la consultation des données historiques.	Moyenne
BF10	Gestion des modèles	Permettre le réentraînement et l'exploitation des modèles.	Moyenne

2.9.2 Priorisation des exigences

Les exigences ont été classées selon leur importance pour le fonctionnement global de la solution.

Les fonctionnalités de priorité **haute** constituent le noyau de Droppy. Elles comprennent notamment la collecte des données, le prétraitement, la détection des anomalies, la détection des fuites, les alertes et la supervision.

Les fonctionnalités de priorité **moyenne** viennent compléter le système et permettent d'améliorer les capacités d'analyse et d'exploitation des données.

Cette priorisation permet de concentrer les efforts de développement sur les fonctionnalités essentielles avant d'intégrer les fonctionnalités complémentaires.

2.10 Critères de réussite

La réussite de la plateforme Droppy est évaluée selon plusieurs dimensions.

2.10.1 Critères fonctionnels

Le système doit être capable :

- ▶ de recevoir correctement les données de télémétrie ;
- ▶ de traiter et préparer les données ;
- ▶ d'identifier les comportements atypiques ;
- ▶ de détecter les situations pouvant correspondre à des fuites ;
- ▶ de produire des prévisions ;
- ▶ de générer et transmettre les alertes ;
- ▶ d'afficher les résultats dans le tableau de bord.

2.10.2 Critères techniques

La solution doit également respecter plusieurs critères techniques :

- ▶ communication correcte entre les différents services ;
- ▶ fonctionnement des échanges temps réel ;
- ▶ persistance fiable des informations ;
- ▶ possibilité de réentraîner les modèles ;
- ▶ modularité des composants ;
- ▶ présence de mécanismes de test et de validation.

2.10.3 Critères liés à l'utilisateur

Enfin, le système doit permettre à l'opérateur :

- ▶ d'accéder rapidement aux informations importantes ;
- ▶ d'identifier facilement les anomalies ;
- ▶ de consulter l'historique ;
- ▶ de comprendre les résultats produits ;
- ▶ de réagir rapidement lorsqu'une alerte critique est générée.

2.11 Conclusion

Ce chapitre a permis de transformer la problématique générale de la détection des fuites d'eau en un ensemble d'exigences fonctionnelles et non fonctionnelles concrètes.

L'analyse du domaine métier a montré l'importance de disposer d'une surveillance continue capable d'exploiter les données issues des équipements IoT. L'étude de l'existant a également mis en évidence les limites des inspections périodiques et des approches reposant uniquement sur des seuils fixes.

La vision fonctionnelle de Droppy repose ainsi sur une chaîne complète allant de la collecte des télémetries jusqu'à leur analyse et leur restitution dans une interface de supervision. Cette chaîne combine traitement des données, règles métier, apprentissage automatique, prévision et notifications.

Les acteurs du système ainsi que les besoins fonctionnels, les besoins non fonctionnels et les contraintes ont ensuite été identifiés. Le cahier des charges obtenu constitue la base de la phase de conception.

Le chapitre suivant sera consacré à la **conception de la solution**. Il présentera l'architecture globale de Droppy, les interactions entre ses différents composants, les flux de données ainsi que les modèles permettant de formaliser la conception du système.

Conception et architecture de la solution Droppy

3.1 Introduction

Après l'analyse des besoins présentée dans le chapitre précédent, cette étape consiste à concevoir la solution Droppy et à définir son architecture.

La conception est présentée à travers plusieurs vues complémentaires permettant de représenter le système sous différents angles : fonctionnel, architectural, dynamique, statique et physique.

Les modèles retenus sont le diagramme de cas d'utilisation, l'architecture globale, deux diagrammes de séquence, le diagramme de classes et le diagramme de déploiement.

3.2 Architecture globale

Droppy repose sur une architecture distribuée organisée autour de trois composants principaux : une application web Angular 19, un Core Service Spring Boot 3 et un microservice d'intelligence artificielle développé avec FastAPI.

Le Core Service constitue le point central de communication entre l'interface utilisateur, le service d'intelligence artificielle et la base de données. L'application Angular communique avec Spring Boot à travers des API REST et un canal WebSocket pour les notifications en temps réel. Spring Boot communique quant à lui avec le microservice FastAPI pour les opérations de détection et de prévision.

Architecture de l'application Droppy

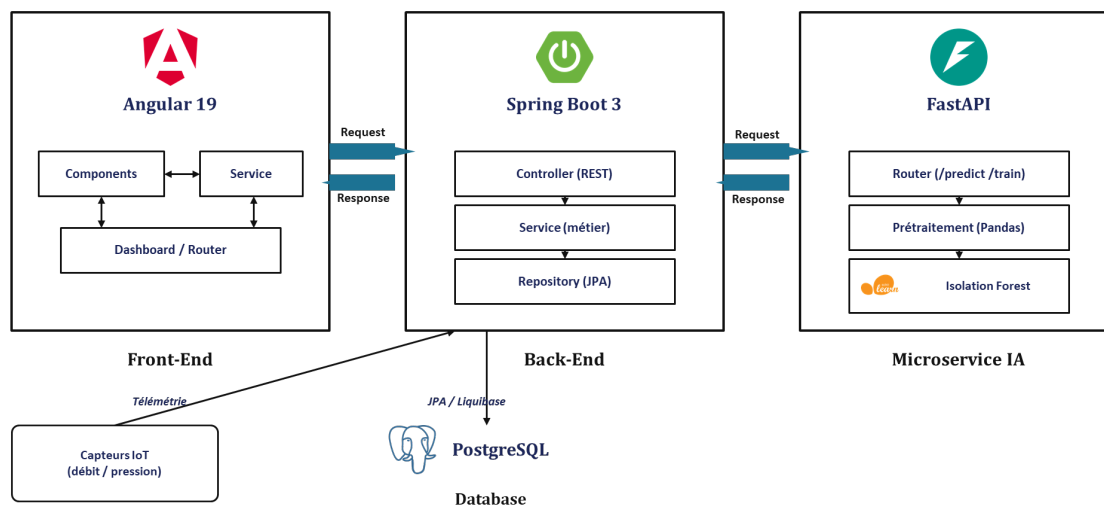


Figure 3.1. Architecture globale de la solution Droppy

Cette architecture permet de découpler l'interface, la logique métier et les traitements d'intelligence artificielle, tout en conservant un point d'orchestration central.

3.3 Modélisation fonctionnelle

Le diagramme de cas d'utilisation présente les principales interactions entre l'utilisateur et la solution Droppy.

L'opérateur constitue l'acteur principal du système. Il utilise le dashboard pour consulter les anomalies, visualiser les données, demander une analyse, consulter les prévisions et traiter les anomalies détectées.

Les traitements automatiques sont également représentés afin de montrer que certaines opérations, notamment l'analyse périodique des dispositifs, sont déclenchées sans intervention directe de l'utilisateur.

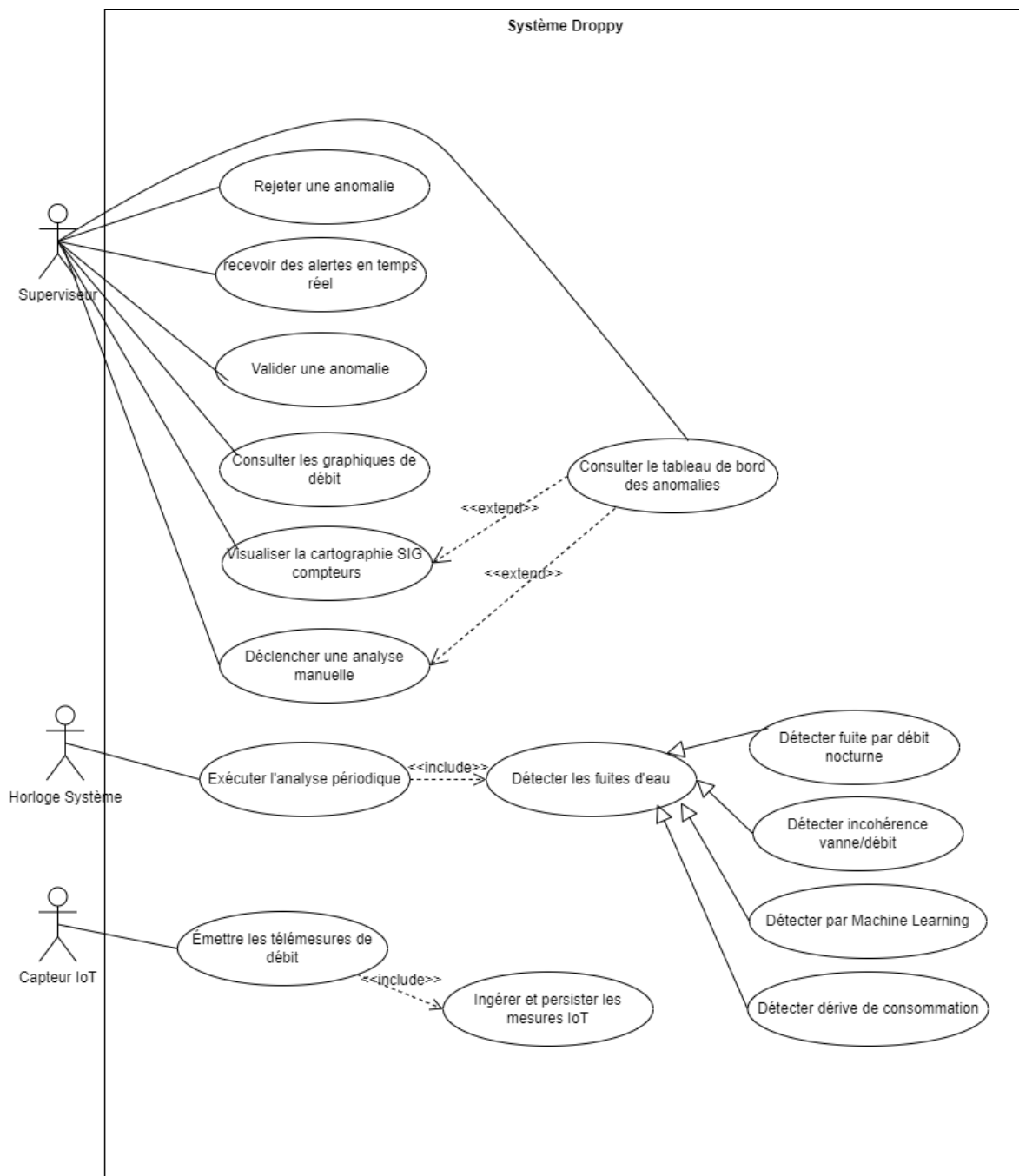


Figure 3.2. Diagramme de cas d'utilisation de Droppy

Ce modèle fournit ainsi une vue fonctionnelle synthétique des principales interactions avec la plateforme.

3.4 Modélisation dynamique

Les diagrammes de séquence permettent de représenter les échanges entre les différents composants lors de l'exécution des scénarios principaux de Droppy.

Deux scénarios ont été retenus : la détection d'une anomalie et la prévision de consommation.

3.4.1 Détection d'une anomalie

Le premier scénario représente le processus de détection d'une anomalie. L'analyse est déclenchée par le scheduler puis orchestrée par le Core Service, qui sollicite le microservice FastAPI.

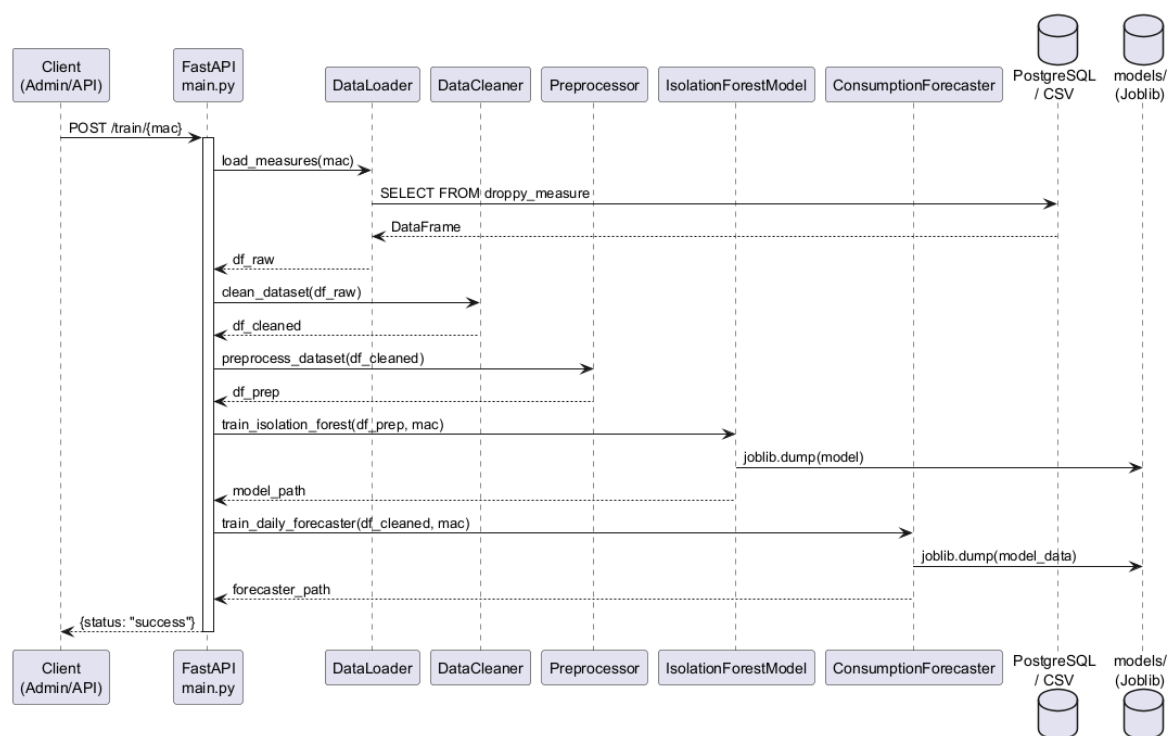


Figure 3.3. Séquence de détection d'une anomalie

Le résultat retourné par le service IA est traité par Spring Boot avant d'être enregistré et transmis au dashboard lorsque cela est nécessaire.

3.4.2 Prévision de consommation

Le second scénario décrit la demande d'une prévision depuis le dashboard. La requête traverse le Core Service avant d'être transmise au microservice FastAPI, qui exécute le modèle de prévision.

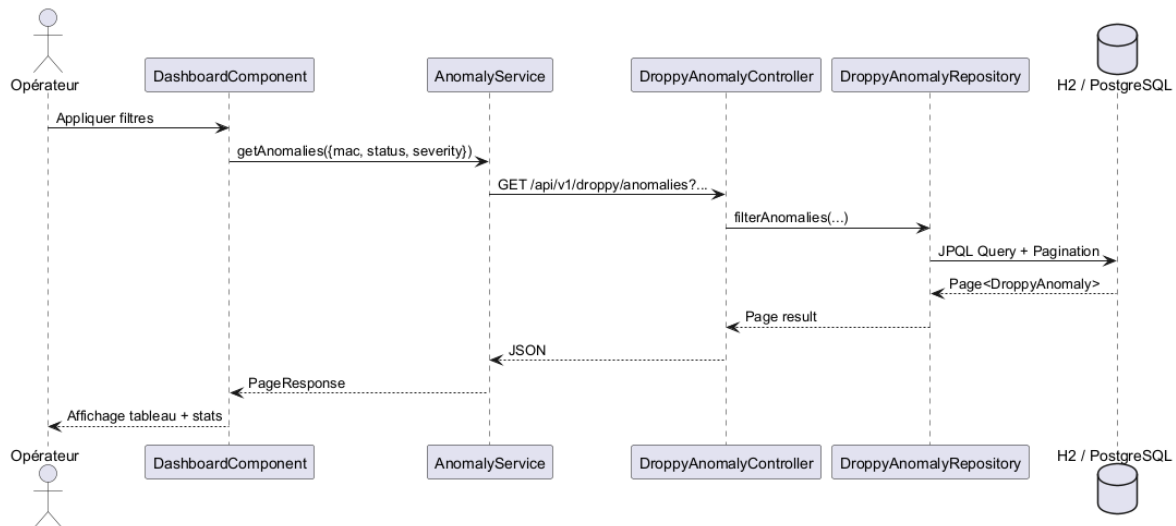


Figure 3.4. Séquence de prévision de consommation

Le résultat est ensuite retourné au dashboard afin d'être présenté à l'opérateur sous une forme graphique.

3.5 Modélisation statique

Le diagramme de classes présente la structure des principaux éléments du Core Service et leurs relations.

La conception suit une séparation des responsabilités entre les contrôleurs, les services métier, les repositories et les entités persistantes. La classe **DroppyAnomaly** constitue l'entité centrale pour la représentation et le suivi des anomalies détectées.

Diagramme de Classes - Entités Droppy (Domain Model)

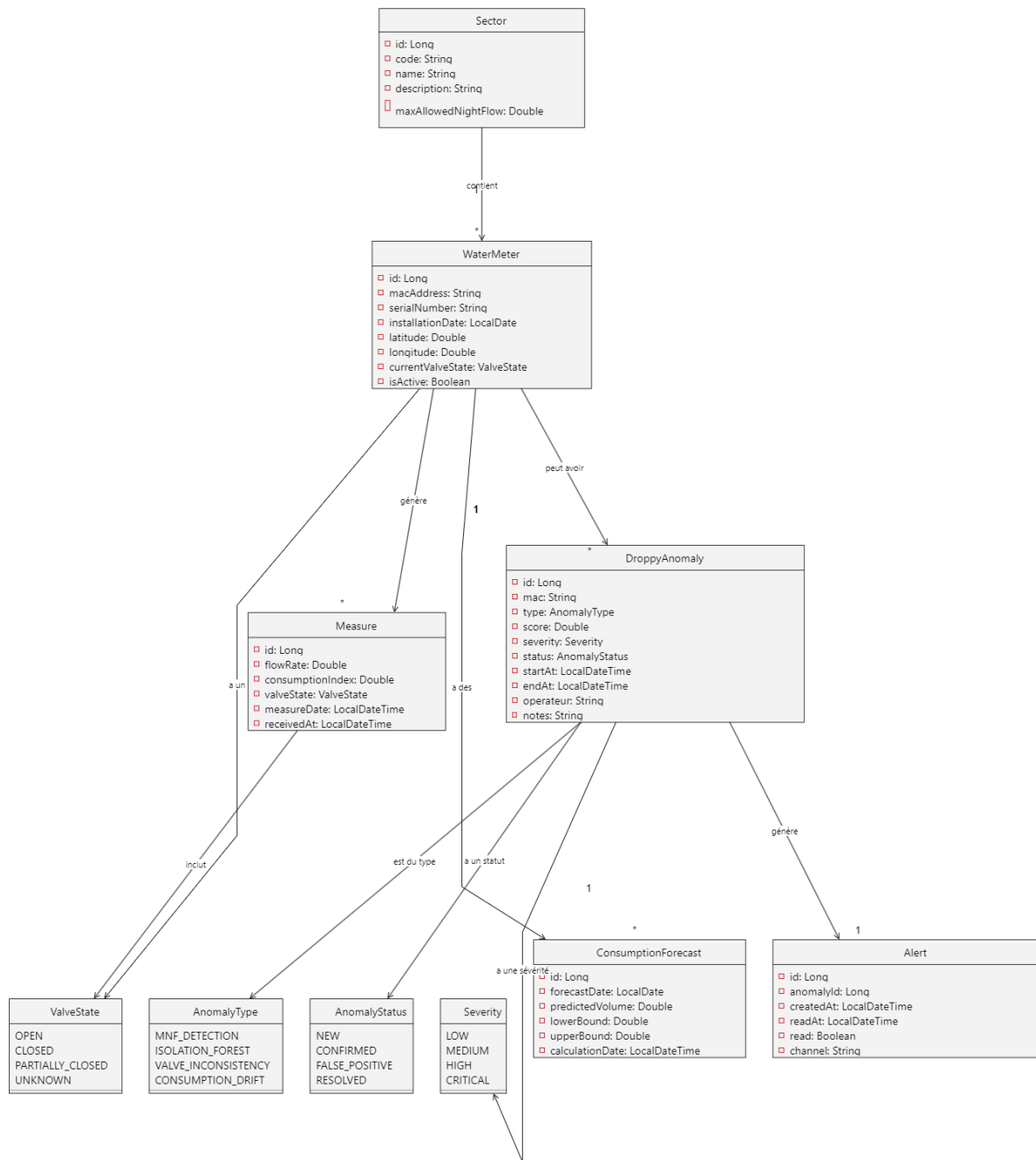


Figure 3.5. Diagramme de classes principal de Droppy

Cette organisation permet de maintenir une séparation claire entre l'exposition des API, la logique métier et l'accès aux données.

3.6 Conclusion

Ce chapitre a présenté la conception de Droppy à travers plusieurs vues complémentaires.

L'architecture globale a permis de définir l'organisation des principaux services, tandis que les diagrammes de cas d'utilisation et de séquence ont permis de représenter les interactions et les scénarios essentiels. Le diagramme de classes a décrit la structure statique du système et le diagramme de déploiement a présenté son organisation dans l'environnement d'exécution.

Cette conception constitue la base de la réalisation technique présentée dans le chapitre suivant.

Réalisation et présentation de l'application Droppy

4.1 Introduction

Après avoir présenté les besoins et la conception de la solution, ce chapitre est consacré à la présentation de l'application Droppy réalisée.

L'objectif est de présenter les principales interfaces de la solution ainsi que les fonctionnalités accessibles à travers le dashboard. Les différentes captures d'écran permettent d'illustrer concrètement le résultat obtenu et le parcours d'utilisation de l'application.

4.2 Présentation générale de l'application

4.2.1 Écran principal

L'écran principal constitue le point d'entrée de l'application. Il offre une vue synthétique des informations importantes et permet à l'utilisateur d'accéder rapidement aux différentes fonctionnalités de supervision.

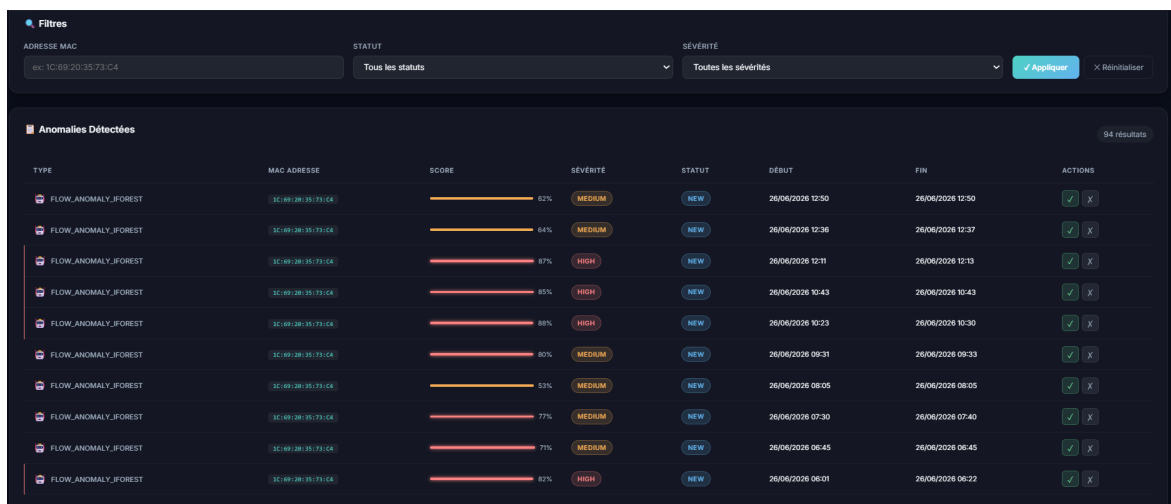


Figure 4.1. Écran principal du dashboard Droppy

4.2.2 Organisation de l'interface

L'interface a été conçue afin de regrouper les informations essentielles dans une présentation claire et directement exploitable par l'utilisateur.

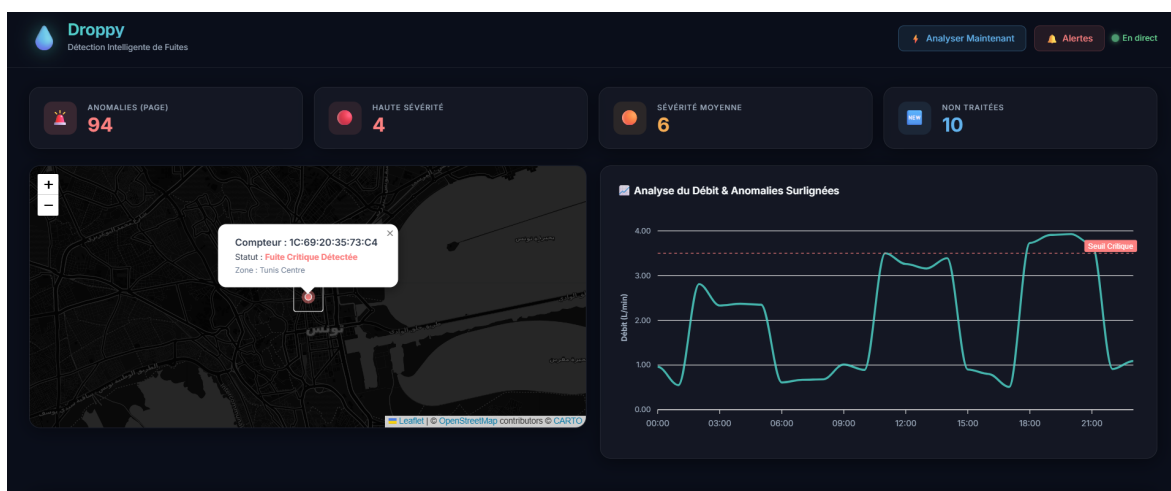


Figure 4.2. Organisation générale de l'interface

4.3 Visualisation des données

4.3.1 Graphiques de suivi

Les graphiques permettent de visualiser l'évolution des données et de faciliter l'interprétation des différents événements observés.

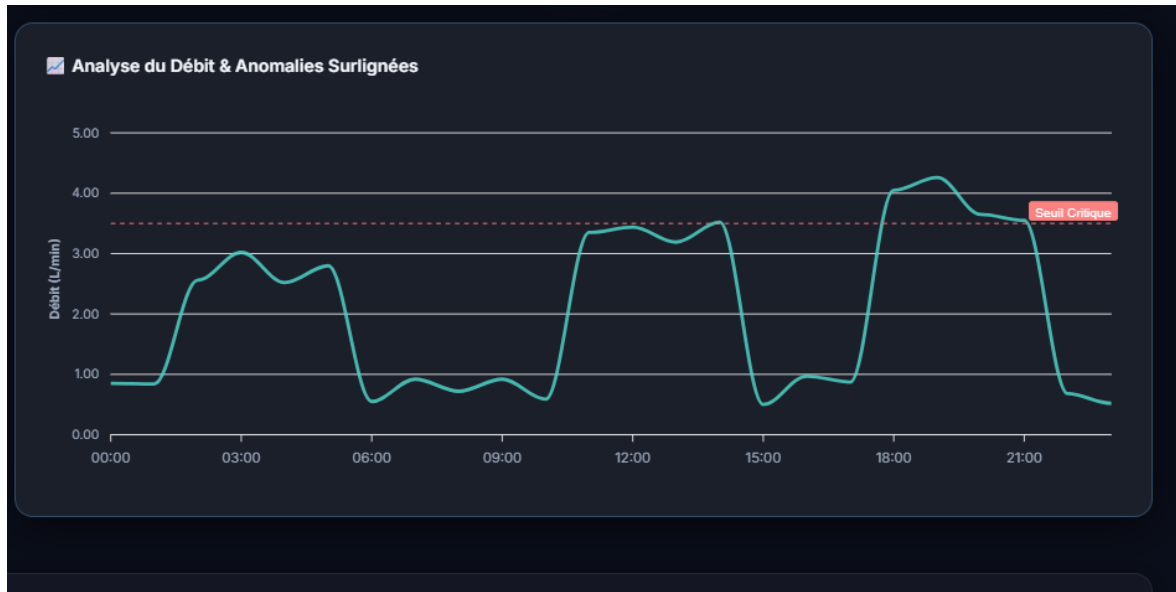


Figure 4.3. Visualisation graphique des données

4.3.2 Visualisation cartographique

La carte permet de localiser les événements associés aux dispositifs supervisés et d'offrir une représentation géographique des anomalies.

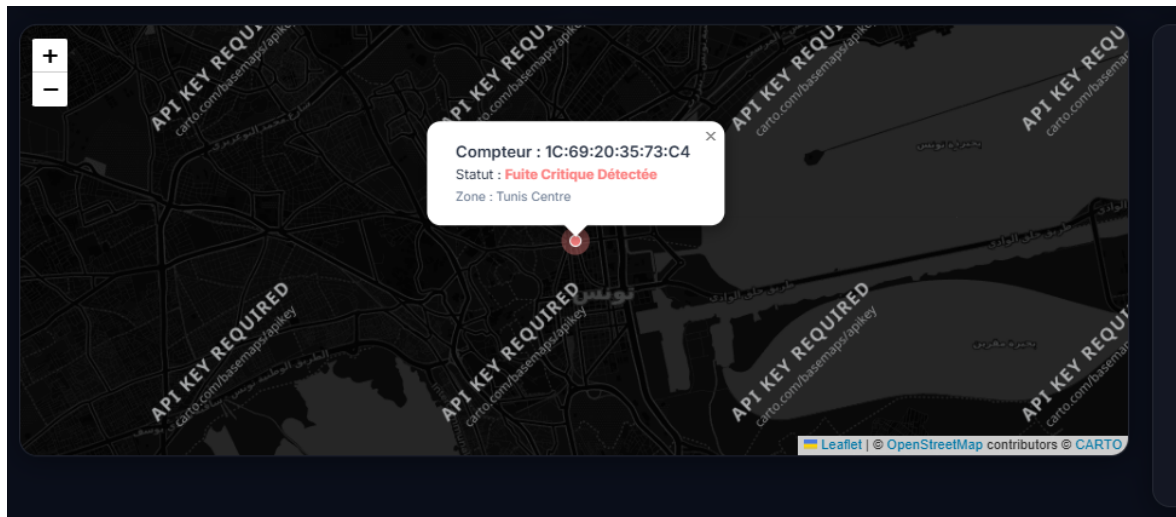


Figure 4.4. Visualisation cartographique des anomalies

4.4 Analyse intelligente

4.4.1 Déclenchement d'une analyse

L'utilisateur peut lancer une analyse depuis l'interface afin d'obtenir une évaluation de l'état d'un dispositif.

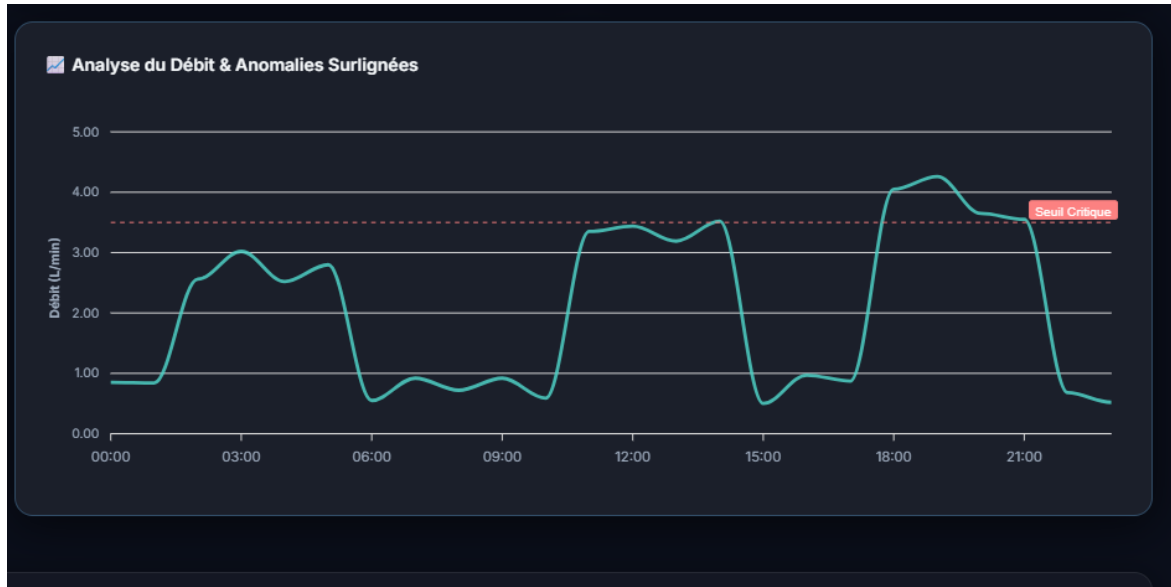


Figure 4.5. Déclenchement d'une analyse

4.5 Conclusion

Ce chapitre a présenté la réalisation de l'application Droppy à travers ses principales interfaces et fonctionnalités.

Les différentes captures permettent d'illustrer concrètement le résultat final de la solution, depuis la supervision générale jusqu'à la consultation des anomalies, la visualisation des données, l'analyse et la prévision.

Le chapitre suivant sera consacré aux tests et à la validation de la solution.

Intelligence artificielle et détection des anomalies

5.1 Introduction

L'intelligence artificielle constitue l'un des éléments centraux de la solution Droppy. Elle permet d'analyser automatiquement les données de télémétrie afin d'identifier des comportements inhabituels et d'estimer l'évolution future de la consommation.

L'approche retenue ne repose pas sur un unique mécanisme. Le service d'intelligence artificielle combine des règles métier avec des méthodes d'apprentissage automatique et de prévision. Cette combinaison permet d'exploiter à la fois les connaissances liées au fonctionnement du réseau et les comportements observés dans les données.

Le pipeline développé dans Droppy peut être résumé comme suit :

**Données → Nettoyage → Prétraitement → Détection → Agrégation →
Résultat**

5.2 Pipeline de traitement des données

Avant l'application des mécanismes de détection, les données sont chargées, nettoyées puis préparées pour les différents traitements.

Le microservice IA sépare cette chaîne en plusieurs étapes correspondant au chargement des mesures, au nettoyage et au prétraitement des données. Cette organisation permet de disposer d'un flux de traitement homogène avant l'exécution des différents mécanismes d'analyse.

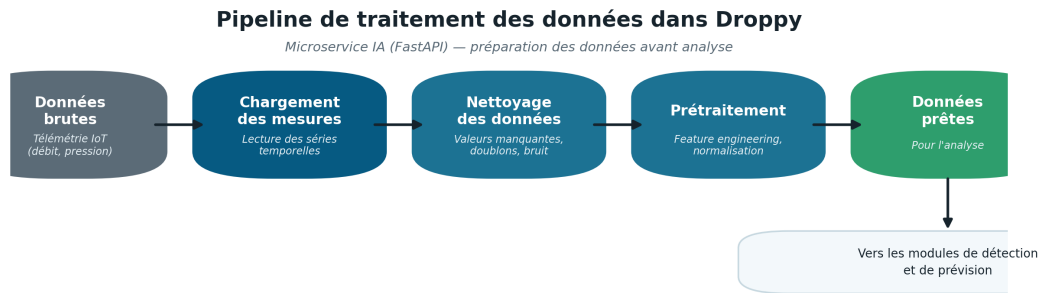


Figure 5.1. Pipeline de traitement des données dans Droppy

Cette séparation permet également de réutiliser les mêmes étapes de préparation pour les différents mécanismes d'analyse.

5.3 Stratégie de détection des anomalies

La détection mise en œuvre dans Droppy repose sur une approche hybride. Lorsqu'une prédiction est demandée, le microservice exécute plusieurs mécanismes indépendants puis regroupe leurs résultats.

Quatre sources de détection sont actuellement intégrées :

- ▶ la détection du *Minimum Night Flow* ;
- ▶ la détection d'incohérences liées aux vannes ;
- ▶ la détection statistique par *Isolation Forest* ;
- ▶ la détection d'écarts de consommation à partir du modèle de prévision.

Les résultats sont ensuite regroupés et les événements correspondant au même type et au même intervalle temporel sont dédoublonnés.

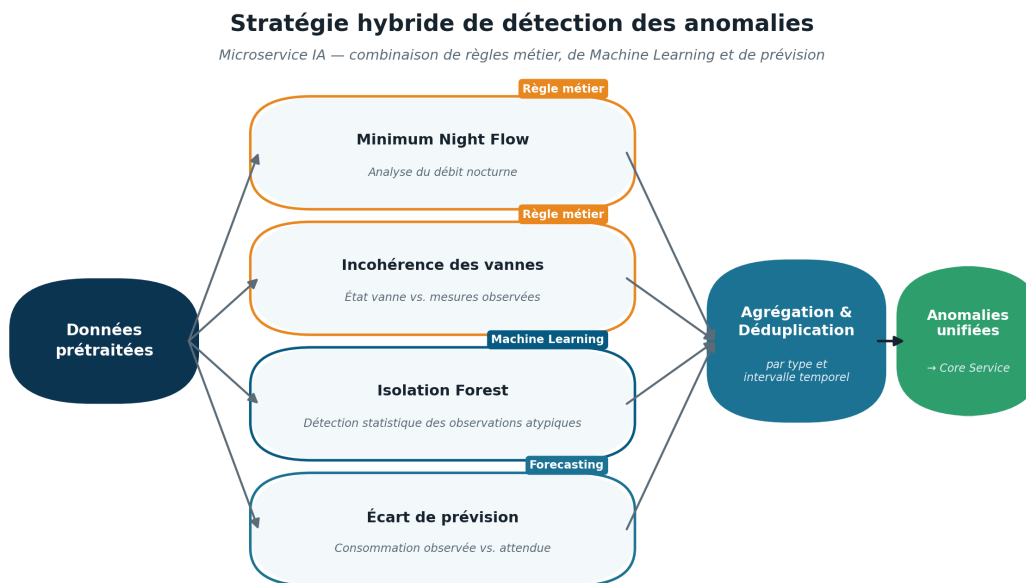


Figure 5.2. Stratégie hybride de détection des anomalies

5.4 Détection basée sur les règles métier

5.4.1 Minimum Night Flow

Le premier mécanisme repose sur l'analyse du débit pendant les périodes nocturnes. L'objectif est d'identifier des situations dans lesquelles le comportement observé s'écarte du fonctionnement attendu pendant ces périodes.

Cette approche permet d'intégrer directement une connaissance métier dans le processus de détection, sans dépendre exclusivement d'un modèle d'apprentissage automatique.

5.4.2 Incohérence liée aux vannes

Le second mécanisme recherche des incohérences entre l'état d'une vanne et le comportement observé dans les mesures.

Cette règle permet de détecter des situations dans lesquelles les mesures observées ne correspondent pas au fonctionnement attendu du dispositif.

5.5 Détection par Isolation Forest

En complément des règles métier, Droppy utilise l'algorithme *Isolation Forest* pour identifier automatiquement les observations qui présentent un comportement inhabituel.

Contrairement à une classification supervisée classique, cette méthode est adaptée à la détection d'observations atypiques. Le modèle est entraîné à partir des données préparées puis utilisé lors de l'analyse d'un dispositif.

Dans l'implémentation de Droppy, le modèle est chargé pour le dispositif concerné et appliqué aux données prétraitées. Le résultat produit contribue ensuite à la liste globale des anomalies.

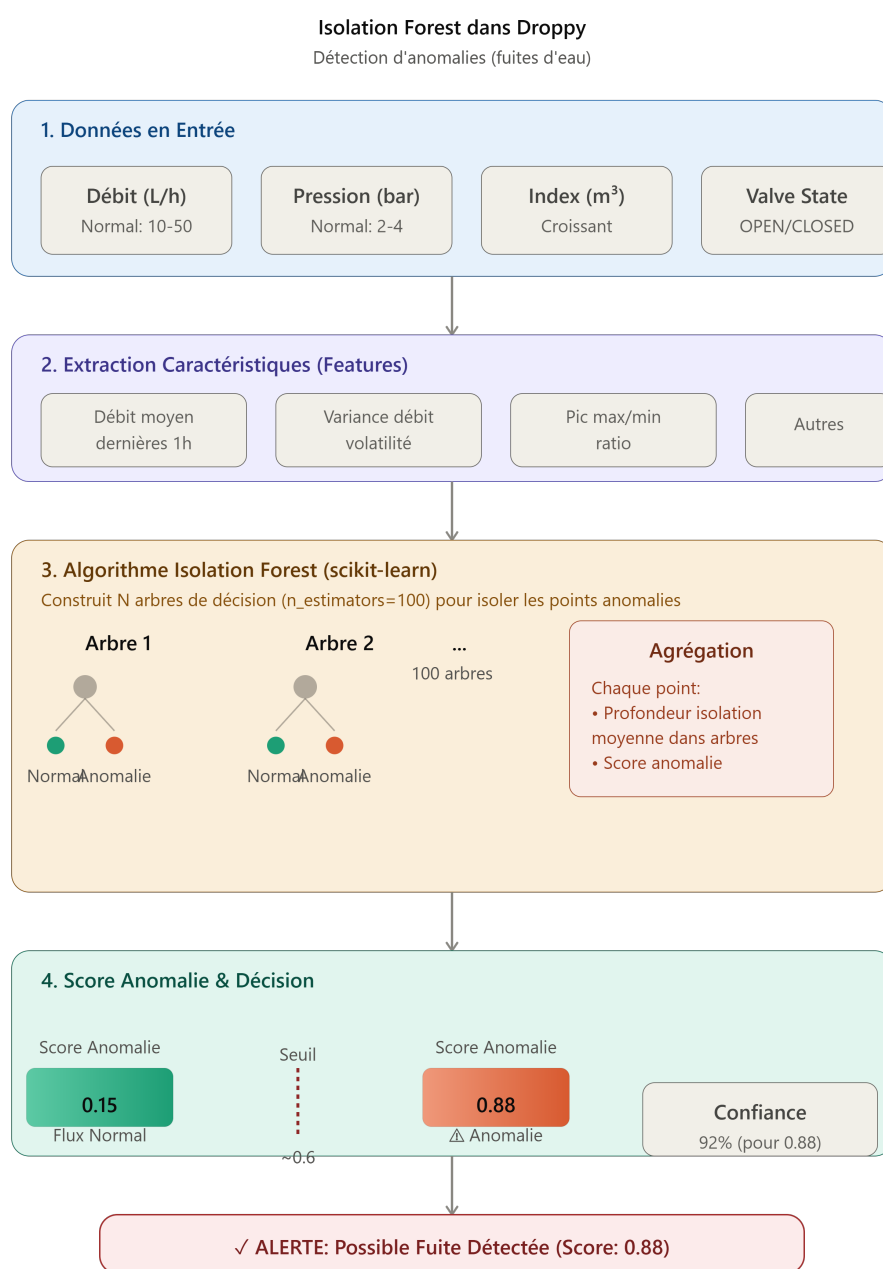


Figure 5.3. Principe d'utilisation de l'Isolation Forest dans Droppy

5.6 Agrégation des résultats

L'une des caractéristiques importantes de l'approche adoptée est la combinaison des différentes sources de détection.

Pour une même période d'analyse, les résultats issus des règles métier, de l'Isolation Forest et de l'analyse de déviation de consommation sont regroupés dans une même réponse.

Le système applique ensuite une étape de déduplication afin d'éviter de retourner plusieurs fois un même événement lorsqu'il est identifié par plusieurs mécanismes.

Cette stratégie permet de produire une sortie unifiée destinée au Core Service et, finalement, au dashboard Dropdy.

5.7 Résultat de l'analyse

Chaque anomalie produite par le microservice contient plusieurs informations permettant son exploitation par les composants du système.

Le résultat de prédiction comprend notamment :

- ▶ le type de l'anomalie ;
- ▶ le score associé à la détection ;
- ▶ l'intervalle temporel concerné ;
- ▶ le niveau de sévérité.

Ces informations sont ensuite transmises au Core Service afin d'être intégrées dans le système de supervision.

5.8 Exposition des fonctionnalités IA

Les fonctionnalités d'intelligence artificielle sont exposées à travers le microservice FastAPI.

Trois opérations principales sont proposées :

- ▶ `/train/{mac}` : entraînement ou réentraînement des modèles associés à un

dispositif;

- ▶ `/predict/{mac}` : exécution de la détection combinant les différents mécanismes d'analyse;
- ▶ `/forecast/{mac}` : génération des prévisions de consommation.

Cette séparation permet au Core Service de solliciter indépendamment les opérations d'entraînement, de détection et de prévision.

5.9 Synthèse

L'intelligence de Droppy repose ainsi sur une combinaison de plusieurs mécanismes complémentaires.

Les règles métier permettent d'intégrer les connaissances liées au fonctionnement du réseau, tandis que l'Isolation Forest apporte une capacité de détection automatique des comportements atypiques. Le modèle de prévision complète cette approche en permettant à la fois d'estimer la consommation future et d'identifier des écarts par rapport au comportement attendu.

Cette approche hybride constitue le cœur analytique de Droppy et permet de transformer les données de télémétrie en événements exploitables par la plateforme de supervision.

5.10 Conclusion

Ce chapitre a présenté le fonctionnement de la composante d'intelligence artificielle de Droppy.

La solution combine un pipeline de préparation des données, des règles métier, l'algorithme Isolation Forest et un mécanisme de prévision de consommation. Les résultats issus de ces différents traitements sont agrégés afin de produire une vision unifiée des anomalies détectées.

Le chapitre suivant sera consacré aux tests et à la validation de la solution afin d'évaluer le comportement des différents composants et les résultats obtenus.

Conclusion générale et perspectives

6.1 Conclusion générale

Ce projet de stage a porté sur la conception et la réalisation de Droppy, une solution intelligente dédiée à la supervision et à l'analyse des données issues d'un réseau d'eau.

L'objectif principal était de proposer une plateforme capable d'exploiter les données de télémétrie afin d'identifier automatiquement des comportements anormaux et de fournir des informations utiles à la supervision du réseau.

La réalisation du projet a permis de mettre en place une architecture distribuée intégrant une interface de supervision, un service central d'orchestration et un microservice dédié aux traitements d'intelligence artificielle. Cette organisation a permis de séparer les différentes responsabilités tout en assurant leur communication.

La conception de la solution a été formalisée à travers plusieurs modèles UML permettant de représenter les interactions, l'architecture, les séquences principales, la structure des classes et l'organisation du déploiement.

La réalisation de l'application a ensuite permis de concrétiser ces choix à travers un dashboard destiné à la supervision des anomalies, à la visualisation des données et à la consultation des prévisions.

La composante intelligente constitue également un élément essentiel de Droppy. L'approche retenue combine plusieurs mécanismes complémentaires, notamment des règles métier, l'Isolation Forest et la prévision de consommation. Cette combinaison permet d'obtenir une analyse plus adaptée aux caractéristiques des données étudiées.

Ainsi, ce projet a permis de mettre en pratique différentes compétences liées au développement logiciel, à la conception d'architectures distribuées, au traitement des données et à l'intelligence artificielle, tout en répondant à une problématique concrète de supervision.

6.2 Apports du projet

La réalisation de Droppy a permis d'aboutir à une solution intégrant plusieurs fonctionnalités complémentaires :

- ▶ supervision des anomalies détectées ;
- ▶ visualisation des informations à travers un dashboard ;
- ▶ analyse automatique des comportements inhabituels ;
- ▶ génération de prévisions de consommation ;
- ▶ transmission des événements en temps réel ;
- ▶ centralisation des résultats pour leur exploitation par l'utilisateur.

Au-delà des fonctionnalités développées, ce projet a également constitué une expérience permettant de mieux comprendre les contraintes liées à l'intégration de plusieurs services logiciels au sein d'une même solution.

6.3 Limites de la solution

Malgré les fonctionnalités réalisées, certaines améliorations restent possibles.

Les performances des mécanismes de détection dépendent notamment de la qualité et de la disponibilité des données historiques. La diversité des comportements observés sur différents dispositifs peut également nécessiter une adaptation des modèles et des paramètres utilisés.

Par ailleurs, l'évolution de la solution vers un environnement de production à grande échelle nécessiterait de renforcer certains aspects liés au déploiement, à la supervision des services et à la gestion d'un nombre plus important de dispositifs.

6.4 Perspectives

Plusieurs évolutions peuvent être envisagées afin d'améliorer les capacités de Droppy.

6.4.1 Amélioration des modèles d'intelligence artificielle

Une première perspective consiste à enrichir les modèles d'analyse en utilisant davantage de données historiques et en expérimentant différentes approches d'apprentissage automatique afin d'améliorer la détection des comportements atypiques.

6.4.2 Extension de la supervision

La solution pourrait être étendue afin de prendre en charge un nombre plus important de dispositifs et de réseaux. Une gestion plus avancée des dispositifs permettrait également d'adapter les analyses aux caractéristiques de chaque installation.

6.4.3 Amélioration du système d'alertes

Le système de supervision pourrait être complété par différents canaux d'alerte afin de permettre une notification plus rapide des événements importants.

6.4.4 Déploiement dans un environnement Cloud

Une évolution vers une infrastructure Cloud permettrait de faciliter le déploiement, la disponibilité et la montée en charge de la solution.

6.4.5 Développement d'une application mobile

Une application mobile pourrait également être développée afin de permettre aux opérateurs de consulter les anomalies et les alertes depuis leurs appareils mobiles.

6.4.6 Évolution vers une maintenance prédictive

À plus long terme, les données historiques et les résultats des modèles pourraient être exploités afin d'aller au-delà de la simple détection d'anomalies et de proposer une approche de maintenance prédictive.

6.5 Conclusion finale

Le développement de Droppy a permis de mettre en œuvre une solution complète associant supervision, analyse de données, intelligence artificielle et visualisation.

Le projet constitue une première base pour une plateforme pouvant évoluer vers une supervision plus automatisée et plus prédictive des réseaux d'eau.

Les perspectives identifiées offrent ainsi plusieurs possibilités d'évolution, aussi bien sur le plan fonctionnel que sur le plan technique. Elles permettraient notamment d'améliorer la précision des analyses, d'étendre la solution à davantage de dispositifs et de faciliter son exploitation dans un environnement réel à grande échelle.

Bibliographie

- [1] Pedregosa, F. et al. (2011).
Scikit-learn : Machine Learning in Python.
Journal of Machine Learning Research, 12, pp. 2825–2830.
- [2] Liu, F. T., Ting, K. M. et Zhou, Z.-H. (2008).
Isolation Forest.
Proceedings of the 2008 IEEE International Conference on Data Mining (ICDM).
- [3] Ramírez, S.
FastAPI Documentation.
<https://fastapi.tiangolo.com/>
- [4] VMware.
Spring Boot Reference Documentation.
<https://spring.io/projects/spring-boot>
- [5] Google.
Angular Documentation.
<https://angular.dev/>
- [6] International Water Association.
Minimum Night Flow for Leak Detection.
Best Practice Guidelines.
- [7] Groupe SFM.
SFM Connect – Solutions IoT Énergie & Eau.
<https://linkedin.com/company/groupe-sfm>

Annexes

A.1 User Stories implémentées

Table A.1. Correspondance User Stories / modules

US	Description	Module
US-05	Health check	FastAPI
US-06	Connexion SQLAlchemy	loader.py
US-07	Prétraitement	preprocessor.py
US-08	Minimum Night Flow	mnf.py
US-09	Isolation Forest	isolation_forest.py
US-10	Valve Inconsistency	valve_inconsistency.py
US-11	Prédiction agrégée	main.py
US-12	Prévision consommation	forecaster.py

A.2 Configuration de déploiement

Table A.2. Services et ports

Service	Port	Commande
FastAPI	8001	uvicorn src.main:app -port 8001
Spring Boot	8082	mvn spring-boot:run
Angular	4200	ng serve -port 4200

A.3 Structure du projet source

```

1 stage/
2 |-- core-service/           # Spring Boot Backend
3 |   |-- src/main/java/      # Controllers, Services, Models

```

```

4 | |-- pom.xml
5 | '-- db/changelog/      # Liquibase
6 |-- dashboard-angular/  # Angular 19 UI
7 | |-- src/app/components/
8 | |-- src/app/services/
9 | '-- package.json
10 |-- src/                # FastAPI AI Engine
11 | |-- data/             # loader, cleaner
12 | |-- features/         # preprocessor
13 | |-- models/           # ML detectors
14 | '-- main.py
15 |-- tests/              # pytest
16 |-- models/             # Joblib serialized models
17 |-- requirements.txt
18 '-- run_app.bat

```

Listing A.1. Arborescence du projet Droppy